

第四章 第二节 Python进阶篇——面向对象：类继承

- **PYRHON 类继承**

继承是面向对象程序设计的重要概念之一。

通过继承，我们可以从既有的类上衍生出新的类。如果程序的需求为仅修改或删除某项功能，此时不需要将该类的成员数据及成员函数重新写一遍，只需要“继承”原先已定义好的类就可以产生新的类了。

继承是指将现有类的属性和行为，经过修改或重写（Override）之后，就可产生出拥有新功能的类，这样可以大幅提升程序代码的可重用性（Reusability）。

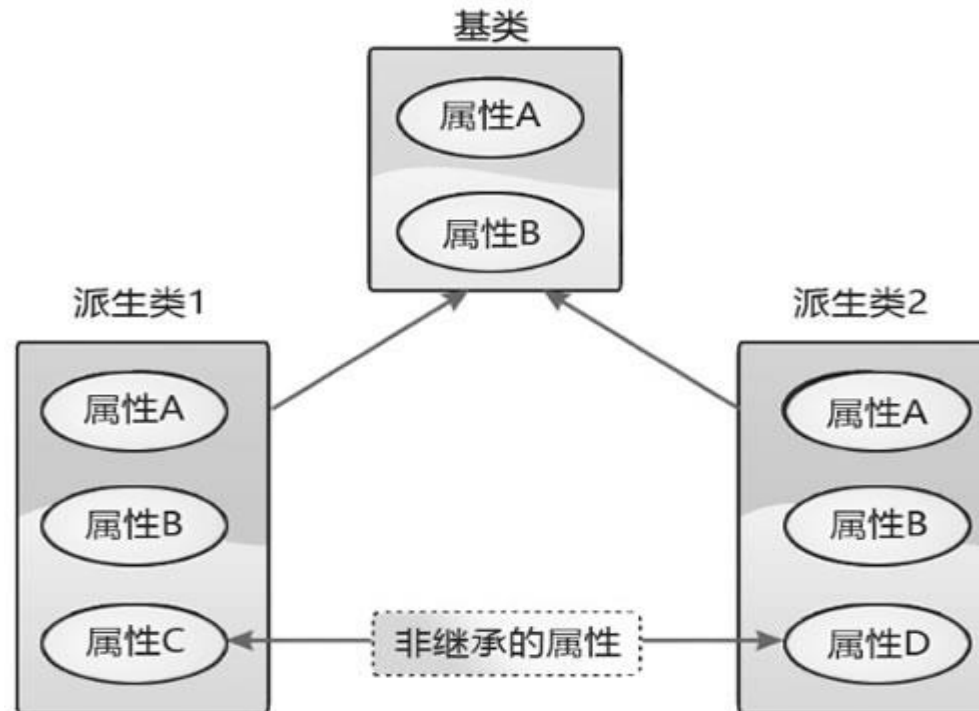
事实上，继承除了可重复利用之前开发过的类之外，最大的优势在于能够维持对象封装的特性。因为继承时不易改变已经设计完整的类，这样可以减少继承时类设计上的错误发生。

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

在 Python 中，在继承之前，原先已建立好的类称为基类（Base Class），而经由继承所产生的新类就称为派生类（Derived Class）。

类之间如果要有继承的关系，必须先建立好基类，也就是父类（Super-class），然后派生类，也就是子类（Sub-class），其相互间的关系如图 1 所示。



第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

所谓单继承（Single Inheritance），即派生类直接继承单独一个基类的成员数据与成员函数。在 Python 中使用继承机制定义子类的语法格式如下。

```
class ChildClass(ParentClass):
```

代码块

首先我们来看单继承的实例，下面这个程序示例会先定义 MobilePhone 基类，接着会以继承的语法去定义 HTC 派生类。

【示例1】单继承的实例。python代码如下：

```
class MobilePhone: #基类
    def touch(self):
        print('我能提供屏幕触控操作的功能')
class HTC(MobilePhone): #派生类
    pass
#产生子类对象
u11 = HTC()
u11.touch()
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

程序解说：

第 1~3 行：定义 MobilePhone 基类。

第 5~8 行：定义 HTC 派生类。

第 9 行：产生 u11 子类的对象。

第 10 行：调用继承自 MobilePhone 基类的touch()方法。

其实子类还可以扩展父类的方法，而不仅仅是照单全收该方法的功能，我们可以将上例修改为以下程序示例。这个示例中除了调用原来父类的方法，还根据自己的需求扩展了子类的方法。

【示例2】在子类中扩展父类的方法。python代码如下：

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

【示例2】在子类中扩展父类的方法。python代码如下：

```
class MobilePhone: #基类
    def touch(self):
        print('我能提供屏幕触控操作的功能')
class HTC(MobilePhone): #派生类
    def touch(self):
        MobilePhone.touch(self)
        print('我也能提供多点触控的操作方式')
#产生子类对象
u11 = HTC()
u11.touch()
```

输出结果：

我能提供屏幕触控操作的功能
我也能提供多点触控的操作方式

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

程序解说：

第 1~3 行：定义 MobilePhone 基类。

第 5~8 行：定义 HTC 派生类，但在这个派生类中扩展父类的 touch() 方法。

第 11 行：产生 u11 子类的对象。

第 12 行：u11 子类对象调用继承自 MobilePhone 基类的 touch() 方法。

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

用 `super()` 函数调用父类的方法

如果子类要调用父类所定义的方法，则必须使用内部函数 `super()`，接下来我们通过一个实例来帮助读者理解相应的概念。

【示例3】在子类中调用父类的方法。python代码如下：

#在子类中调用父类的方法—使用`super()`函数

```
class Weekday(): #父类
    def display(self, pay):
        self.price=pay
        print('欢迎来购物')
        print('购买总金额{:,}元'.format(self.price))

class Holiday(Weekday): #子类
    def display(self, pay): #重写display方法
        super().display(pay)
        if self.price >= 15000:
            self.price *= 0.8
        else:
            self.price
        print('8折 {:,}元'.format(self.price))

monday = Weekday() #父类对象
monday.display(25000)
Christmas = Holiday() #子类对象
Christmas.display(18000)
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

输出结果：

```
欢迎来购物
购买总金额25,000元
欢迎来购物
购买总金额18,000元
8折 14,400.0元
```

程序解说：

第 3~7 行：定义 Weekday 父类。

第 9~16 行：定义 Holiday 的子类，其中第 11 行的 `super()` 方法表示在子类中调用父类的方法。

第 18~19 行：实例化父类的对象，接着以此对象调用类中的 `display()` 方法。

第 21~22 行：实例化子类的对象，接着以此对象调用类中的 `display()` 方法。

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

用 `super()` 函数调用父类的方法

如果子类要调用父类所定义的方法，则必须使用内部函数 `super()`，接下来我们通过一个实例来帮助读者理解相应的概念。

【示例4】使用 `super()` 函数调用父类的方法，对子类来说，即使在 `__init__()` 方法内也同样适用。python代码如下：

```
#调用__init__()方法
class Animal():#父类
    def __init__(self):
        print('我属于动物类')
class Human(Animal): #子类
    def __init__(self, name):
        super().__init__()
        print('我也属于人类')
man = Human('黄种人')#子类对象
```

输出结果：

```
我属于动物类
我也属于人类
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

程序解说：

第 3~5 行：定义 Animal 父类。

第 7~10 行：定义 Human 子类，在第 9 行用 `super()` 方法调用父类的 `__init__()` 方法来进行初始化的部分工作。

第 12 行：实例化子类的对象。

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

获取兄弟类的属性

假设有一个父类 Tom，它有两个子类 Andy 和 Michael，Andy 和 Michael 这两个类称为兄弟类。

如果 Andy 类想取得 Michael 兄弟类的 height 属性，可以使用下列语法。

```
Michael().height #Andy取得Michael 兄弟类的height属性
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

【示例5】设计 3 个类，并取得兄弟类的属性。python代码如下：

```
class Tom():#父类
    def __init__(self):
        self.height1=178
class Andy(Tom):#父类是Tom
    def __init__(self):
        self.height2=180
        super().__init__()
class Michael(Tom):#父类是Tom
    def __init__(self):
        self.height3=185
        super().__init__()
def display(self):
    print('父亲Tom 的身高:', self.height1,'厘米')
    print('兄弟Andy的身高:', Andy().height2,'厘米')
    print('自己Michael的身高:', self.height3,'厘米')
m1=Michael()
m1.display()
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 单继承与定义子类

输出结果：

父亲Tom 的身高: 178 厘米

兄弟Andy的身高: 180 厘米

自己Michael的身高: 185 厘米

程序解说：

第 1~3 行：定义 Tom 父类。

第 5~8 行：定义 Andy 子类。

第 10~17 行：定义 Michael 子类，其中第 16 行的“`Andy().height2`”就是在 Michael 类中调用其兄弟类 Andy 的属性。

第 19 行：实例化 Michael 类的对象 m1。

第 20 行：调用 Michael 类的 `display()` 方法。

第四章 第二节 Python进阶篇——面向对象：类继承

- 多继承与定义子类

派生类只有一个基类时称为“单继承”；当基类有两个及以上时就称作“多继承”（Multiple Inheritance），我们以逗号分隔这些基类。

多继承是指派生类继承自多个基类，而这些被继承的基类相互之间可能都没有关系，简单地说，就是直接继承了两个或两个以上的基类。

多继承声明语句的语法如下。

```
class ChildClass(ParentClass1, ParentClass2,...):
```

程序代码块

我们将举两个简单的实例来示范 Python 的多继承，第一个例子祖父类的两个派生类中的方法名称相同。

读者可以留意一下，在这个例子中设定了一个 Mermaid 类的对象 Alice，再由这个对象分别调用 feature1()、feature2()、feature3() 3 个方法，但因为其父类一 Human 及父类二 Fish 这两个类同时拥有 feature2() 方法，读者可以试着观察到底会执行哪一个 feature2() 方法。

第四章 第二节 Python进阶篇——面向对象：类继承

- 多继承与定义子类

【示例6】多继承示例一。python代码如下

#多继承示例

```
class Animal: #祖父类
    def feature1(self):
        print('大多数动物能自发且独立地移动')
class Human(Animal): #父类一
    def feature2(self):
        print('人类是一种有思考能力与情感的高级动物')
class Fish(Animal): #父类二
    def feature2(self):
        print('水生脊椎动物的总称')
class Mermaid(Human, Fish): #子类同时继承两个种类
    def feature3(self):
        print('又称人鱼,传说中的生物, 同时具备人及鱼的部分特性')
#产生子类实例
alice = Mermaid()
alice.feature1()
alice.feature2()
alice.feature3()
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 多继承与定义子类

输出结果：

大多数动物能自发且独立地移动

人类是一种有思考能力与情感的高级动物

又称人鱼,传说中的生物，同时具备人及鱼的部分特性

程序解说：

第 3~5 行：定义 Animal 祖父类。

第 7~9 行：定义 Human 的父类一。

第11~13 行：定义 Fish 的父类二。

第15~17 行：子类 Mermaid 同时继承 Human 父类一及 Fish 父类二。

第 20 行：产生 Alice 子类实例。

第 21 行：调用继承自 Animal 祖父类的 feature1() 方法。

第 22 行：调用继承自 Human 父类一的 feature2() 方法。

第 23 行：调用继承自 Mermaid 类的 feature3() 方法。

第四章 第二节 Python进阶篇——面向对象：类继承

- 多继承与定义子类

上述例子中的 Human 和 Fish 类同时拥有 feature2() 方法，读者应该注意到最后只会执行第一个继承的父类的 feature2() 方法。

但是如果我们将 Human 和 Fish 类的方法取不一样的名字，当上述程序在建立 Mermaid 类的 Alice 对象时，就会分别调用多继承的父类中的不同方法。

以下程序示例只将上例中原本名称相同的 feature2() 方法改成了不同的名称，读者可以清楚分辨出这两个程序不同的输出结果。

第四章 第二节 Python进阶篇——面向对象：类继承

- 多继承与定义子类

【示例7】多继承示例二。python代码如下：

#多继承范例2

```
class Animal: #祖父类
    def feature1(self):
        print('大多数动物能自发且独立地移动')
class Human(Animal): #父类一
    def feature2(self):
        print('人类是一种有思考能力与情感的高级动物')
class Fish(Animal): #父类二
    def feature3(self):
        print('水生脊椎动物的总称')
class Mermaid(Human, Fish): #子类同时继承两种类
    def feature4(self):
        print('又称人鱼,传说中的生物, 同时具备人及鱼的部分特性')
#产生子类实体
alice = Mermaid()
alice.feature1()
alice.feature2()
alice.feature3()
alice.feature4()
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 多继承与定义子类

输出结果：

大多数动物能自发且独立地移动
人类是一种有思考能力与情感的高级动物
水生脊椎动物的总称
又称人鱼,传说中的生物, 同时具备人及鱼的部分特性

程序解说：

第 3~5 行：定义 Animal 祖父类。

第 7~9 行：定义 Human 的父类一。

第 11~13 行：定义 Fish 的父类二，此处方法名称和父类一要改成不同的名称，笔者取名为 feature3()。

第 15~17行：子类 Mermaid 同时继承 Human 父类一及 Fish 父类二，此处方法名称取为 feature4()。

第 20 行：产生 Alice 子类实体。

第 21 行：调用继承自 Animal 祖父类的 feature1() 方法。

第 22 行：调用继承自 Human 父类一的 feature2() 方法。

第 23 行：调用继承自 Fish 父类二的 feature3() 方法。

第 24 行：调用继承自 Mermaid 类的 feature4() 方法。

第四章 第二节 Python进阶篇——面向对象：类继承

- 重写基类方法

当我们从基类继承所有的成员后，在原先的基类内可能有某些成员函数不符合程序的需要。

事实上，不一定所有继承的成员都必须照单全收，用户可以在派生类中以相同名称、相同参数以及相同的返回值的方法来取代基类的方法。利用这种方式来建立派生类成员函数的动作称为重写。

简单来说，重写就是重新改写所继承的父类的方法，但不会影响到父类中原来的方法。以下程序示范了如何在子类中重写父类的方法。

第四章 第二节 Python进阶篇——面向对象：类继承

- 多继承与定义子类

【示例8】重写的实例。python代码如下：

#在子类中重写父类的方法

```
class Normal(): #父类
```

```
    def subsidy(self, income):
```

```
        self.money = income
```

```
        if self.money >= 500000:
```

```
            print('小康家庭补助金额: ', end = ' ')
```

```
            return 5000
```

```
class Poor(Normal): #子类
```

```
    def subsidy(self, income): #重写subsidy方法
```

```
        self.money = income
```

```
        if self.money < 300000:
```

```
            print('中低收入家庭补助金额: ', end = ' ')
```

```
            return 10000
```

```
student1 = Normal() #建立父类对象
```

```
print(student1.subsidy(780000), '元')
```

```
student2 = Poor() #建立子类对象
```

```
print(student2.subsidy(250000), '元')
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 多继承与定义子类

输出结果：

小康家庭补助金额： 5000 元

中低收入家庭补助金额： 10000 元

程序解说：

第 2~7 行：定义 Normal 父类。

第 9~14 行：定义 Poor 子类，这个子类会重写 subsidy() 方法。

第 16~17 行：建立 student1 父类对象，并调用父类中的 subsidy() 方法，再将其结果输出。

第 19~20 行：建立 student2 子类对象，并调用子类中重写的 subsidy() 方法，再将其结果输出。

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

在 Python 中与继承相关的函数有 `isinstance()` 及 `issubclass()`。

函数 `isinstance()`

`isinstance()` 函数是 Python 中的一个内置函数，包含两个参数，第一个参数是对象，第二个参数是类。其语法如下。

`isinstance( 对象, 类)`

这个函数的功能是判断第一个参数的对象是否属于第二个参数类的一种。如果第一个参数的对象属于第二个参数类或其子类，会返回 `True`，否则返回 `False`。

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

【示例9】沿用多继承的例子，测试并输出一系列的 `isinstance()` 函数的返回值。python代码如下：

```
class Animal: #祖父类
    def feature1(self):
        print('大多数动物能自发且独立地移动')
class Human(Animal): #父类一
    def feature2(self):
        print('人类是一种有思考能力与情感的高级动物')
class Fish(Animal): #父类二
    def feature3(self):
        print('水生脊椎动物的总称')
class Mermaid(Human, Fish): #子类同时继承两种类
    def feature4(self):
        print('又称人鱼,传说中的生物, 同时具备人及鱼的部分特性')
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

【示例9】沿用多继承的例子，测试并输出一系列的 `isinstance()` 函数的返回值。python代码如下：

```
#产生子类实体
tiger = Animal()
daniel= Human()
goldfish=Fish()
alice = Mermaid()

print("tiger是属于Animal类:",isinstance(tiger,Animal))
print("daniel是属于Animal类:",isinstance(daniel,Animal))
print("goldfish是属于Animal类:",isinstance(goldfish,Animal))
print("alice是属于Animal类:",isinstance(alice,Animal))
print("=====")
print("tiger是属于Human类:",isinstance(tiger,Human))
print("daniel是属于Human类:",isinstance(daniel,Human))
print("goldfish是属于Human类:",isinstance(goldfish,Human))
print("alice是属于Human类:",isinstance(alice,Human))
print("=====")
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

【示例9】沿用多继承的例子，测试并输出一系列的 `isinstance()` 函数的返回值。python代码如下：

```
print("tiger是属于Fish类:", isinstance(tiger, Fish))
print("daniel是属于Fish类:", isinstance(daniel, Fish))
print("goldfish是属于Fish类:", isinstance(goldfish, Fish))
print("alice是属于Fish类:", isinstance(alice, Fish))
print("=====")
print("tiger是属于Mermaid类:", isinstance(tiger, Mermaid))
print("daniel是属于Mermaid类:", isinstance(daniel, Mermaid))
print("goldfish是属于Mermaid类:", isinstance(goldfish, Mermaid))
print("alice是属于Mermaid类:", isinstance(alice, Mermaid))
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

【示例9】沿用多继承的例子，测试并输出一系列的 `isinstance()` 函数的返回值。python代码如下：

输出结果：

```
tiger是属于Animal类: True
daniel是属于Animal类: True
goldfish是属于Animal类: True
alice是属于Animal类: True
```

```
=====
```

```
tiger是属于Human类: False
daniel是属于Human类: True
goldfish是属于Human类: False
alice是属于Human类: True
```

```
=====
```

```
tiger是属于Fish类: False
daniel是属于Fish类: False
goldfish是属于Fish类: True
alice是属于Fish类: True
```

```
=====
```

```
tiger是属于Mermaid类: False
daniel是属于Mermaid类: False
goldfish是属于Mermaid类: False
alice是属于Mermaid类: True
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

【示例9】沿用多继承的例子，测试并输出一系列的 `isinstance()` 函数的返回值。python代码如下：

程序解说：

第1~3行：定义Animal祖父类。

第5~7行：定义Human父类一。

第9~11行：定义Fish父类二。

第13~15行：子类同时继承两个类，即父类一及父类二。

第18~21行：产生子类对象。

第22~40行：测试并输出一系列的 `isinstance()` 函数的返回值。

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

函数issubclass()

issubclass() 函数是 Python 中的一个内置函数，包含两个参数，其语法如下。

issubclass(子类,父类)

这个内置函数的功能是如果类 1 是类 2 所指定的子类，则返回 True，否则返回 False。请看下面的程序范例。

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

函数issubclass()

【示例10】 内置函数 `issubclass()` 的实例。python代码如下：

```
class Animal: #祖父类
    def feature1(self):
        print('大多数动物能自发且独立地移动')
class Human(Animal): #父类一
    def feature2(self):
        print('人类是一种有思考能力与情感的高级动物')
class Fish(Animal): #父类二
    def feature3(self):
        print('水生脊椎动物的总称')
class Mermaid(Human, Fish): #子类同时继承两种类
    def feature4(self):
        print('又称人鱼,传说中的生物,同时具备人及鱼的部分特性')
print('又称人鱼,传说中的生物,同时具备人及鱼的部分特性')
print("Mermaid属于Fish子类:",issubclass(Mermaid,Fish))
print("Mermaid属于Human子类:",issubclass(Mermaid,Human))
print("Mermaid属于Animal子类:",issubclass(Mermaid,Animal))
```

第四章 第二节 Python进阶篇——面向对象：类继承

- 继承相关函数

函数issubclass()

输出结果：

Mermaid属于Fish子类: True

Mermaid属于Human子类: True

Mermaid属于Animal子类: True

程序解说：

第 1~3 行：定义 Animal 祖父类。

第 5~7 行：定义 Human 父类一。

第 9~11 行：定义 Fish 父类二。

第 13~15 行：子类同时继承两个类，即父类一及父类二。

第 17 行：判断 Mermaid 是否为 Fish 的子类，并将其结果值输出。

第 18 行：判断 Mermaid 是否为 Human 的子类，并将其结果值输出。

第 19 行：判断 Mermaid 是否为 Animal 的子类，并将其结果值输出。